

Sistema de Gestión de Flota Espacial

Objetivo

Diseñar un sistema para una agencia espacial que gestione diferentes tipos de naves. Debes aplicar los pilares de la POO para evitar la duplicidad de código y proteger la integridad de los datos de navegación.

Estructura de Paquetes

1. **com.agencia.espacial.naves:** Contendrá la jerarquía de las naves.
2. **com.agencia.espacial.main:** Contendrá la lógica de ejecución.

Definición de Clases y Atributos

1. Clase Abstracta: NaveEspacial

- **Atributos:**
 - private String nombre: Nombre de la nave.
 - private double combustible: Nivel actual (0 a 100).
 - protected int capacidadPasajeros: Número máximo de personas.
- **Métodos:**
 - Constructor: Que inicialice nombre y capacidad. El combustible siempre empieza en 100.
 - public abstract void viajar(int distancia): Define cómo se desplaza la nave.
 - public void recargarCombustible(): Rellena el atributo combustible a 100.
 - Getters/Setters: Implementar para todos los atributos con **validación** en el combustible (no puede ser menor a 0 ni mayor a 100).

2. Clase: NaveCarga

- **Atributos:**
 - private int cargaMaxima: Peso máximo en toneladas.
- **Métodos:**
 - viajar(int distancia): Resta combustible según la fórmula: $\text{distancia} * 0.5$. Si el combustible es insuficiente, muestra un error.

3. Clase: NaveExploracion

- **Atributos:**
 - private int cantidadSensores: Número de dispositivos de escaneo.
- **Métodos:**
 - viajar(int distancia): Resta combustible según la fórmula: $\text{distancia} * 0.2$ (son más ligeras).
 - public void escanearPlaneta(): Imprime un mensaje indicando cuántos sensores están activos.

4. Clase: EstacionEspacial (clase independiente)

- **Atributos:**
 - private String ubicacion: Coordenadas de la base.
 - private int nivelEnergia: Energ a de la base.
- **M todos:**
 - public void anclarNave(NaveEspacial nave): Recibe cualquier objeto que herede de NaveEspacial (Polimorfismo) y muestra su nombre.

5. Clase: NaveCombate

- **Atributos:**
 - private int escudoEnergia: Nivel de protecci n extra.
- **M todos:**
 - viajar(int distancia): Consume distancia * 0.8 de combustible.
 - public void activarEscudos(): Imprime el estado del escudo.

Requerimientos de Implementaci n

1. **Protecci n de Datos:** Ninguna clase externa debe poder modificar el combustible de una nave directamente. Todo debe pasar por los m todos definidos.
2. **L gica de Paquetes:** Aseg rate de que las clases est n en sus carpetas correspondientes y utiliza los import necesarios en la clase Main.
3. **Prueba de Polimorfismo:** En el Main, crea un array de tipo NaveEspacial[] que contenga una nave de cada tipo. Usa un bucle para que todas "viajen" 50 unidades.  Qu  pasa con el combustible?